2M Engineering

PCardio

API

Document Title:	PCardio - API
Company:	2M Engineering ltd – John F Kennedylaan 3, 5555XC, Valkenswaard, The Netherlands
Project:	PCardio
Author:	Jose Lopez Hidalgo

Contents

Contents 2
1 Introduction 4
2 Communication Protocols 5
2.1 BLE Communication 5
2.2 Serial Communication 6
3 Core Interface 7
3.1 Read Packet 7
3.2 Data Packet 8
3.3 Abort Packet 8
3.4 Stream Packet 8
4 Serial Interface 9
4.1 Transmit 9
4.2 Receive 9
4.3 Upgrade 9
4.4 Reboot 10
5 Memory Map 11
5.1 Info Memory 12
5.1.1 Id 12
5.1.2 Address 12
5.1.3 Version 12
5.1.4 Battery 12
5.1.5 Device Status 12
5.1.6 Input error code 12
5.1.7 Input error timestamp 12
5.1.8 Hardware Version 13
5.1.9 Hardware Variant 13
5.2 Device Configuration Memory 13
5.2.1 Pin 13
5.2.2 Time 13
5.2.3 Button mode 13
5.2.4 Advertising mode 14
5.2.5 Advertising time 14
5.3 Measuring Configuration Memory 14
5.3.1 User id 14
5.3.2 Start time 14
5.3.3 Stop time 14
5.3.4 Sampling configuration 14
5.4 Stream Memory 14
6 Data Packet 15
6.1 Header 15
6.1.1 Time 15
6.1.2 Ticks 15
6.1.3 Status 15
6.1.4 Payload size 16
6.1.5 Checksum 16
6.2 Payload 16

7 Sampling configuration 17
7.1 Sensor configuration 17
8 File interface 20
8.1 Configuration file 20
8.2 Data files 20
8.3 Log file 21
9 Input error handling 22
10 Good practices 23
10.1 Reading/writing multiple fields 23
10.2 Single operations 23
10.3 Battery and data size monitoring 23
10.4 Quick Start 23
11 Experimental Sensors 25

1 Introduction

The PCardio BLE API provides developers with powerful tools to access and utilize the diverse capabilities of the PCardio device. This interface facilitates seamless integration of various sensor data—such as ECG, IMU, and more—into custom applications. By enabling users to configure sensor settings and manage data efficiently, the PCardio platform supports innovative solutions in cardio and health sensing products tailored for diverse applications, including medical, sports, and military sectors.

Whether you are building mobile apps, IoT solutions, or any other software that requires seamless connectivity and data exchange, this guide will provide you with the necessary tools and resources to access all the functionalities of the PCardio recorder.

The PCardio recorder can be accessed through two communication protocols: wireless BLE communication, and serial communication accessible through a micro-USB connector. The BLE communication provides access to the Core Interface that focuses on essential device configuration and data access. The serial communication is primarily used for direct PC connectivity and provides access to the Serial Interface, which extends the functionality of the Core Interface. Additionally, the PCardio recorder exposes a Mass Storage USB interface that allows user to configure the recorder and to download the data.

Throughout this document, you will find detailed explanations on how to:

- Discover and communicate with the PCardio recorder over BLE and serial communication
- Understand the structure of the files in the device's Mass Storage
- Configure device and measurement settings.
- Interact with the device's internal memory map
- Parse the data streamed or logged by the PCardio recorder.

Firmware version supported: v1.1.2 or higher

2 Communication Protocols

To exchange data with the PCardio recorder, you first need to establish a connection using one of its available communication protocols. Only after this connection is made can the interfaces be used to send commands or retrieve information from the device. Each protocol has its own specific characteristics, which are detailed in the following subsections.

2.1 BLE Communication

The PCardio recorder communicates over BLE using the Nordic UART Service (NUS) — a custom BLE service that emulates a serial port. For more details, see Nordic Semiconductor's official NUS documentation.

PCardio exposes a single GATT (Generic Attribute Profile) service. In BLE, the GATT protocol organizes data into services. Each service groups related characteristics. Characteristics represent readable, writable, or notifiable values — effectively, the interface for data exchange. You can read more about this in this link.

The PCardio service is identified by the following 128-bit UUID: 6E400001-B5A3-F393-E0A9-E50E24DCCA9E This UUID is required to discover and connect to the device's custom NUS implementation.

Within this service, PCardio defines two key characteristics:

- RX Characteristic: used to write data to the device (e.g., Core Interface packets).
- TX Characteristic: used to receive data from the device via notifications.

To receive messages from PCardio, notifications must be explicitly enabled on the TX characteristic.

Characteristic	UUID	Properties
RX Characteristic	6E400002-B5A3-F393-E0A9-E50E24DCCA9E	Write, Write Without Response
TX Characteristic	6E400003-B5A3-F393-E0A9-E50E24DCCA9E	Notify

Table 1. BLE Characteristics

When scanning for BLE devices, your application will receive two types of packets: the Advertisement packet and the Scan Response packet. These are broadcast by nearby devices and contain metadata that helps identify them before a connection is made.

The PCardio recorder sends a custom advertisement that includes the device name, "PCardio".

The Scan Response packet includes the service UUID, used to identify its support for the Nordic UART Service (NUS).

The PCardio recorder utilizes the 2 Mbps PHY introduced in BLE 5.0. To take full advantage of this increased throughput and ensure compatibility, we recommend using BLE 5.0 or higher on the central device.

To optimize performance, several BLE parameters should be considered:

- Connection Interval: Defines how often the peripheral and central can exchange data. It can range from 7.5 milliseconds to 4 seconds, in 1.25 milliseconds increments. Actual limits vary by operating system and BLE stack.
- ATT MTU (Attribute Maximum Transmission Unit): Increases the maximum size of each GATT packet, reducing the overhead for large messages.
- Data Length Extension (DLE): Expands the payload size beyond the default 27-byte BLE packet limit.

Maximizing both ATT MTU and DLE helps ensure that most Core Interface requests fit within a single BLE transaction, improving throughput and reducing latency.

2.2 Serial Communication

To use the PCardio recorder over a serial (USB) connection, start by connecting the device to one of your computer's USB ports using the provided cable.

Once connected, the device will appear in your system's list of serial ports. You can identify it by inspecting the USB device metadata — specifically the bus-reported device description and parent properties.

The relevant identifiers are:

- Product Name: "PCardio"
- VID (Vendor ID): 16D0
- PID (Product ID): 06A1
- MAC Address: A unique hardware identifier that corresponds to the first part of the QR code printed on the back of the sensor

These properties allow your application to reliably detect and distinguish PCardio recorder from other connected USB peripherals.

For a reliable connection, ensure you use a USB cable that supports data transfer—not just charging. Additionally, configure the serial port with the following settings for optimal communication:

- Baud rate: 460800

Parity: None

Stop bits: One

When opening the serial port, make sure to enable Data Terminal Ready (DTR) to properly initialize communication.

For more details on serial communication with PCardio, please refer to the external documentation linked here.

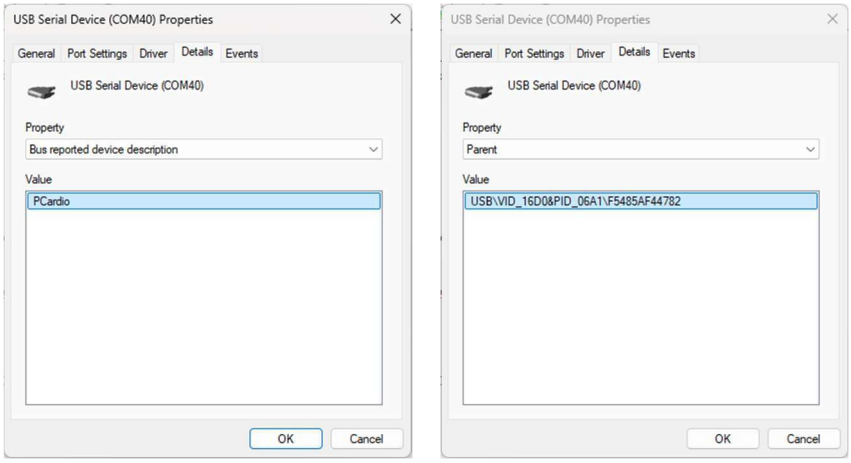


Figure 1. Serial properties of the PCardio recorder

3 Core Interface

The Core Interface provides direct access to the device's memory through four distinct packet types, designed to simplify reading and writing memory blocks:

- Opcode: a single byte that specifies the packet type.
- Address: a 32-bit unsigned integer indicating the starting memory address for the read or write operation.
- Length: a 16-bit unsigned integer specifying how many bytes to read or write.
- Data: a byte array containing the payload, i.e., either the data to be read from the device or the data to be written to the device.

Byte 0	**Byte 1**	**Byte 2**	**Byte 3**	**Byte 4**	**Byte 5**	**Byte 6**	**Byte 7**	**...**	**Byte N**
Opcode	Address				Length		Data		

Table 2. Packet fields and byte order

Not all fields are present in every packet type. Some are optional and can be omitted depending on the operation. The table below outlines which fields are required for each packet type

Packet type	**Fields**			
	Opcode	**Address**	**Length**	**Data**
Read	Used	Used	Used	Not Used
Data	Used	Used	Used	Used
Abort	Used	Not Used	Not Used	Not Used
Stream	Used	Used	Used	Not Used

Table 3. Format for each packet type

NOTE: All multi-byte fields in the packets use little-endian format. Pay close attention to the examples below to see how values are encoded.

NOTE: Large Core Interface messages may be split across multiple smaller packets because of protocol size limits. Your application should concatenate these fragments and wait until the entire message is received before attempting to parse it.

NOTE: The PCardio recorder supports only one operation at a time. You should always wait for the current data transaction to complete before initiating a new one. Avoid sending read or write packets while the device is actively streaming, as this can cause communication issues or data loss.

3.1 Read Packet

Read packets request a specific number of bytes from a designated memory address on the PCardio recorder. Use these packets when you need to retrieve fixed blocks of data.

Byte Address	0	1	2	3	4	5	6	7
0	0x01	0x10	0x00	0x00	0x00	0x04	0x00	

Table 4. Example of read packet to read 4 bytes from memory address 16.

3.2 Data Packet

Data packets are used to write data to the device, allowing you to override parts of its memory. Additionally, the device sends Data packets as responses to Read or Stream requests, delivering the requested data payload.

Byte Address	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0x02	0x10	0x00	0x00	0x00	0x04	0x00	0x01	0x00	0x00	0x00					

Table 5. Example of data packet to write 1 to a 32-bit register at address 16.

3.3 Abort Packet

Abort packets are single-byte commands that interrupt any ongoing data streaming. Use this packet to stop a continuous stream of data from the device immediately.

3.4 Stream Packet

Stream packets request continuous data streaming from the PCardio recorder. Unlike Read packets, they cause the device to send new data continuously as it becomes available. This streaming is restricted to a specific region of the device's memory known as the stream memory.

Byte Address	0	1	2	3	4	5	6	7
0	0x04	0x00	0x00	0x00	0x10	0xED	0x00	

Table 6. Example of a stream packet requesting 237 bytes.

4 Serial Interface

The Serial Interface provides access to the Core Interface over the serial interface and wraps the Core Interface packet in an ASCII encoded message. Besides the Core Interface packet, additional packets can be transmitted and received. The Serial Interface messages follow the following structure:

- Prefix: A Dollar character '$' (hex value 0x24) that marks the start of the packet.
- Opcode: A single character indicating the Serial Interface packet type.
- Data: Contains the encapsulated Core Interface packet as a hex encoded byte array. Every byte consists of two characters between '0..9' or 'A...F', representing hexadecimal notation. The first 2 characters of the Data part in the message represent the 1st byte of the hex encoded byte array. The 3rd and 4th character represent the 2nd byte of the hex encoded byte array, etcetera.
- Postfix: A Line Feed character '\n' (hex value 0x0A) marks the end of the packet.

Byte Address	0	1	2	...	N-1	N
Field	Prefix	Opcode	Data (optional)			Postfix
Character	$	T/R/U				\n

Table 7. Packet fields and byte order of the Serial Interface packets.

4.1 Transmit

This packet allows you to send a Core Interface packet to the device via the serial interface. It uses opcode 'T' (hex value 0x54).
If a response is necessary, the device will reply with a Receive packet.

Byte Address	0	1	2	...	N-1	N
Field	0x24	0x54	Core Interface packet			0x0A
Character	$	T				\n

Table 8. Transmit packet.

4.2 Receive

This packet type is sent by the device in response to a Transmit packet. Its opcode 'R' (hex value 0x52).

Byte Address	0	1	2	...	N-1	N
Field	0x24	0x52	Core Interface packet			0x0A
Character	$	R				\n

Table 9. Receive packet.

4.3 Upgrade

This packet is used to put the device into bootloader mode, which enables firmware updates. The opcode for this packet is 'U' (hex value 0x55). This packet type does not include a data field.

Byte Address	0	1	2	3
Field	0x24	0x55	0x0A	
Character	$	U	\n	

Table 10. Upgrade packet.

4.4 Reboot

This packet is used to hard-reset the PCardio recorder. The opcode for this packet is ‘B’ (hex value 0x42). This packet type does not include a data field.

Byte Address	0	1	2	3
Field	0x24	0x42	0x0A	
Character	$	B	\n	

Table 11. Reboot packet.

5 Memory Map

The PCardio device exposes the following memory map in the BLE interface. It is composed of 4 different memory locations:

- Info memory
- Device configuration memory
- Measurement configuration memory
- Stream memory

Memory Map						
Name	**Address**	**Size**	**Type**	**Read/Write**	**Protected**	**Accessible modes**
Info Memory						
Id	0	8	UInt64	read only	No	
Address	8	8	UInt64	read only	No	
Version	16	4	UInt8[]	read only	No	
Free Space	20	4	UInt32	read only	No	
File Size	24	4	UInt32	read only	No	
Battery	28	4	UInt32	read only	No	
Device Status	32	4	UInt32	read only	No	
Input Error Code	36	4	UInt32	read only	No	
Input Error Timestamp	40	4	UInt32	read only	No	
Hardware Version	44	1	UInt8	read only	No	
Hardware variant	45	1	UInt8	read only	No	
Device Configuration Memory						
Pin	128	4	UInt8[]	read/write	No	All modes
Time	132	4	UInt32	read/write	Yes	All modes
Button mode	136	4	UInt32	read/write	Yes	Idle only
Advertising mode	140	4	UInt32	read/write	Yes	Idle only
Advertising time	144	4	UInt32	read/write	Yes	Idle only
Measurement Configuration Memory						
User Id	256	12	UInt8[]	read/write	Yes	Idle only
Start Time	268	4	UInt32	read/write	Yes	Idle only
Stop Time	272	4	UInt32	read/write	Yes	Idle only
Sampling Configuration	276	68	UInt8[]	read/write	Yes	Idle only
Stream Memory						
Stream data	268435456	16384	UInt8[]	read only	Yes	

Table 12. PCardio memory map.

5.1 Info Memory

This memory location contains read-only registers that provide information to identify the PCardio device, and to know its status.

5.1.1 Id

A unique 64-bit unsigned integer that serves as the device's identifier.

5.1.2 Address

This register holds the device's unique 64-bit MAC address.

NOTE: The QR code on the back of each PCardio recorder combines the address and the first 4 digits of the Id (in hexadecimal), separated by a dash (-).

5.1.3 Version

Represents the firmware version of the device. Details on how to read this version are provided in the table below.

Byte Address	0	1	2	3
0	Major version	Minor version	Revision version	Interface version

Table 13. Structure of the version register.

5.1.4 Battery

Indicates the current battery level of the device as a percentage of the full battery.

5.1.5 Device Status

This register reflects the device measurement and connectivity state

Byte 0								Byte 1	Byte 2	Byte 3
Bit 0	Bit 1	Bit 2	Bit 3	Bit 4	Bit 5	Bit 6	Bit 7			
Logging	Streaming	Docking	Lead-off	Not used				Not used	Not used	Not used

Table 14. Device status register.

5.1.6 Input error code

This register shows the latest input error. See section 8 for a detailed explanation of input error types.

5.1.7 Input error timestamp

This register holds the Unix time in seconds of the latest input error that occurred.

5.1.8 Hardware Version

This register shows the PCB version of the recorder.

Value	Version
0	V2.1
1	V2.2
7	V1.X

Table 15. Hardware version options.

5.1.9 Hardware Variant

This register specifies the hardware variant of the recorder.

Value	Variant
0	EMG
1	1 Lead + Respiration
2	6 Leads

Table 16. Hardware variant options.

5.2 Device Configuration Memory

This memory location contains registers that modify the behaviour of the recorder.

5.2.1 Pin

This register controls access to certain protected registers. Protected registers cannot be read out nor written to unless you first set the pin. To enable access, you must write the following sequence of bytes to this register:

Byte Address	0	1	2	3
0	0x6C	0x6F	0x76	0x65

Table 17. Pin register value.

5.2.2 Time

This register holds the current Unix time in seconds. If the device has been reset or unused for an extended period, it may be necessary to manually set the correct time.

5.2.3 Button mode

This register controls how the long button press affects the measurement status.

Value	Status
0	No action
1	Start logging enabled
2	Start and stop logging enabled

Table 18. Button configuration options.

5.2.4 Advertising mode

This register controls the BLE advertising behaviour.

Value	Status
0	BLE is disabled
1	Starts advertising after USB disconnection only
2	Starts advertising after USB disconnection or button press
3	Advertising always on.

Table 19. Advertising configuration options.

5.2.5 Advertising time

This register sets the BLE advertising duration of the recorder in seconds. The maximum value is 180 seconds.

5.3 Measuring Configuration Memory

This memory location contains registers that determine the state and configuration of the measurement.

5.3.1 User id

This register contains space for up to 12 characters that can be used to identify who the data belongs to.

NOTE: the recorder does not encrypt the value written to the user id register. Therefore, do not write names or other confidential information in plain text. Instead, encrypt it before writing to the PCardio device.

5.3.2 Start time

This register is used to schedule the Unix time in seconds at which the recording should start. The logging will start automatically whenever the Time register is higher than this value.

5.3.3 Stop time

This register is used to schedule the Unix time in seconds at which the recording should stop. The logging will stop automatically whenever the Time register is higher than this value.

5.3.4 Sampling configuration

This register contains the configuration used for the measurement. It allows to enable or disable sensors, set their gain, sample rate and type. See section 5 for a detailed explanation of the structure of this register.

5.4 Stream Memory

This memory location contains the most recent data packet to be streamed. The length of a data packet is dependent on the sampling configuration. The maximum size of a data packet is 16384 bytes.

The format and parsing details of these data packets are described in section 6.

6 Data Packet

The PCardio data packets have a variable size that depends on the sampling configuration. Each packet starts with a 13-byte header, followed by the data payload.

Byte Address	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	Header															
16	Payload															
…																
13+N																

Table 20. General layout of a PCardio data packet.

6.1 Header

The 13-byte header encodes several key fields.

Byte Address	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**
0	Time				Ticks				Status	Payload size		Checksum				

Table 21. Structure of the device state register.

6.1.1 Time

Unix time in seconds of the last sample in the buffer.

6.1.2 Ticks

The number of elapsed ticks from a 32.768 kHz clock that must be added to the timestamp. Since a 32.768 kHz clock produces 32,768 ticks per second, convert this value to standard ticks (100-nanosecond intervals) by multiplying it by $\frac{10000000}{32768}$.

6.1.3 Status

This UInt8 register contains information about the current logging, streaming, connectivity and battery statuses.

Bit 0	**Bit 1**	**Bit 2**	**Bit 3**	**Bit 4**	**Bit 5**	**Bit 6**	**Bit 7**
Logging	Streaming	BLE status		Battery status		Not used	

Table 22. Status field.

BLE status	**Meaning**
0	BLE is off
1	Device is advertising
2	Device is connected

Table 23. BLE status values

Battery status	**Meaning**
0	Battery is between 0 and 25%
1	Battery is between 25 and 50%
2	Battery is between 50 and 75%
3	Battery is between 75 and 100%

Table 24. Battery status values.

6.1.4 Payload size

Indicates the length of the sensor data, in bytes.

6.1.5 Checksum

Used for validating the integrity of the sensor data. It is computed using the CRC-16-CCITT (polynomial 0x1021) with 0xFFFF initial value.

6.2 Payload

The payload contains the data from all the enabled sensors. The order and size of each sensor data block is determined by its sampling rate, data type and position in the sampling configuration. See Sampling Configuration section for an example

7 Sampling configuration

The PCardio device offers extensive customization options for users to tailor their measurements precisely to their needs. Users have control over multiple aspects, including:

- Sensor Selection: choose which sensors are active based on the specific data requirements for each use case.
- Gain Adjustment: set individual gain levels for each sensor to ensure data accuracy and optimize the signal-to-noise ratio.
- Sample Rate Configuration: define the sample rate for each sensor to balance data resolution with resource efficiency.
- Data Type Customization: choose the data format that best suits the application, optimizing for factors like storage or precision.

This flexibility makes the PCardio device adaptable to a range of scenarios, whether for precise diagnostics or research applications requiring tailored data collection.

The configuration is set by modifying the 68-byte sampling configuration register. This register includes a 32-bit unsigned integer that specifies the number of sensors enabled, followed by an array containing up to 16 possible sensor configurations.

Byte Address	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	Sensor count				Config Sensor[0]				Config Sensor[1]				Config Sensor[2]			
…	…															
64	Config Sensor[15]															

Table 25. Sampling configuration register.

7.1 Sensor configuration

The sensor configuration contains 4 unsigned 8-bit fields:

- Sensor ID
- Sensor Gain ID
- Sample rate ID
- Data type ID

Byte Address	0	1	2	3
0	Sensor ID	Sensor Gain ID	Sample Rate ID	Data Type ID

Table 26. Structure of the sensor configuration.

Each sensor supported by the system provides a defined set of valid configuration options. Not all configuration options are available for every sensor; the valid combinations for each sensor are listed in the following tables. In addition, each sensor produces data according to one or more Data Type IDs.

The data type definition specifies the data format, including the number of elements, total size in bytes, whether it is signed or unsigned, physical units, and the corresponding analog and digital value ranges. The tables below summarize the supported configuration identifiers and provide the detailed data type definitions.

Sensor Gain Id	0	1	2	3	4	5	6	7
Gain value	x1	x2	x3	x4	x6	x8	x12	x16

Table 27. Sensor Gain Ids.

Sample Rate ID	0	1	2	3	4	5	6	7	8	9	10
Sample Rate (Hz)	1000	800	500	400	250	200	100	50	25	10	5

Table 28. Sample rate Ids.

Sensor	ID	Sensor Gains	Sample Rates	Data Types
Battery	0	0(×1)	10(5Hz)	0
ECG 1-channel	1	0(×1), 1(×2), 2(×3), 4(×6), 6(x12)	0(1000Hz), 2(500Hz), 4(250Hz)	1
Respiration BioZ	2	0(×1), 1(×2), 2(×3), 4(×6), 6(x12)	0(1000Hz), 2(500Hz), 4(250Hz)	2
ECG 2-channels	3	0(×1), 1(×2), 2(×3), 4(×6), 6(x12)	0(1000Hz), 2(500Hz), 4(250Hz)	3
EMG 2-channels	4	0(×1), 1(×2), 2(×3), 4(×6), 6(x12)	0(1000Hz), 2(500Hz), 4(250Hz)	3
Accelerometer	5	0(×1), 1(×2), 3(×4), 5(×8)	1(800Hz), 3(400Hz), 5(200Hz), 6(100Hz), 7(50Hz)	4
Gyroscope	6	0(×1), 1(×2), 3(×4), 5(×8), 7(x16)	1(800Hz), 3(400Hz), 5(200Hz), 6(100Hz), 7(50Hz)	5, 6
Magnetometer	7	0(x1)	6(100Hz), 7(50Hz)	7
Button Marker	10	0(x1)	10(5Hz)	8
Tap Marker	11	0(x1)	10(5Hz)	8
Lead-off	12	0(x1)	10(5Hz)	8

Table 29. Sensor Ids and their valid configurations.

Data type ID	Units	Total size (bytes)	Elements	Signed	Analog min	Analog max	Digital min	Digital max
0	Percentage	1	1	No	0	100	0	100
1	Millivolts	3	1	Yes	-2420	2420	-8388608	8388607
2	Ohms	3	1	Yes	-80666.67	80666.67	-262144	262143
3	Millivolts	6	2 (Ch1, Ch2)	Yes	-2420	2420	-8388608	8388607
4	Gs	6	3 (X, Y, Z)	Yes	-16	16	-32768	32767
5	Degrees/second	6	3 (X, Y, Z)	Yes	-2000	2000	-28571	28570
6	Rad/second	6	3 (X, Y, Z)	Yes	-34.906585	34.906585	-28571	28570
7	Gauss	6	3 (X, Y, Z)	Yes	-49.125	49.125	-32768	32767
8	Unitless	1	1	No	0	1	0	1

Table 30. Data type Ids and their definitions.

NOTE: the general formula to convert from digital to analog value is:

$$value_A = \left(\frac{max_A - min_A}{max_D - min_D}(value_D - max_D) + max_A\right)/Gain$$

where the subscripts 'A' and 'D' mean analog and digital, respectively.

Example: For a sensor gain of 8 and a default analog range of ±16 the actual range will be ±2.
Since the PCardio device is optimized for throughput, the sampling configuration will determine the size of the data packets. The following pseudocode explains how you can calculate the size of the data packet, and how to find the start index for the data from each of the sensors that are enabled.

```
packet_size = 9

for i = 0 to count do

    start_address[i] = packet_size

    packet_size += sensors[i].data_type.total_size * (sensors[i].sample_rate / 5)
```

Table 31. Pseudocode to calculate packet size and sensor start indices.

NOTE: you can find the total size for each data type in the subsections below. The sample rate needs to be divided by 5 since the PCardio device creates 5 packets per second.

Example: Let's assume that we want to enable the accelerometer, the button marker and the battery level, in that order. The desired configuration for each sensor is:

- Accelerometer: gain of 3, 50Hz of sample rate and measured in Gs.
- Button marker: gain of 1, 5Hz of sample rate and the default/unitless data type.
- Battery level: gain of 1, 5Hz of data and measured in percentage.

With this configuration, the sampling configuration register is:

Count	**Sensor 0**				**Sensor 1**				**Sensor 2**			
	Id	**Gain**	**Sample rate**	**Type**	**Id**	**Gain**	**Sample rate**	**Type**	**Id**	**Gain**	**Sample rate**	**Type**
0x03	0x05	0x02	0x07	0x04	0x0A	0x01	0x0A	0x08	0x00	0x01	0x0A	0x00

Table 32. Sampling configuration example.

This configuration will result in the following data packet, with a length of 75 bytes:

Byte Address	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**
0	Header													Acc[0]		
16				Acc[1]						Acc[2]						
32	Acc[3]					Acc[4]						Acc[5]				
48		Acc[6]						Acc[7]						Acc[8]		
64				Acc[9]						Button Marker	Battery					

Table 33. Data packet example.

NOTE: the order of the sensor data is equivalent to the order of the sensors in the sampling configuration.

8 File interface

When you connect to the PCardio device via the docking station, the file interface becomes available. By using this interface, you can configure the recorder and measurement configurations, and access the recorder data.

There are three different files in this interface: “config.txt”, “data_YYYYMMddHHmmss.txt” and “log.txt”.

8.1 Configuration file

The “config.txt” file contains part of the info memory, recorder configuration memory and measurement configuration memory. It has a fixed length of 144 bytes, and it is required for the PCardio device to work. When the file is missing, the recorder enters an error state, and it can only go back to normal mode when the file is restored.

Modifying this file has the same effect as changing the memory registers via the BLE packets. The new configuration is read when the recorder is disconnected from the docking.

The prefix and postfix contain fixed values, as shown in the table below.

Byte Address	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**
0	Prefix ({0x50, 0x43, 0x41, 0x52, 0x44, 0x49, 0x4F, 0x00})								Id							
16	BLE Address								Version				Button Mode			
32	Advertising Mode				Advertising Time				Unused							
48	Unused				User Id											
64	Start Time				Stop Time				Sampling Configuration							
80																
96																
112																
128													Postfix ({0x0D, 0x0A, 0x0D, 0x0A})			

Table 34. Structure of the configuration file.

8.2 Data files

When the PCardio logs data, it creates a file with a name like “data_” plus the file creation date as “YYYYMMddHHmmss” where:

- YYYY: represents the year
- MM: represents the month
- dd: represents the day
- HH: represents the hours, from 0 to 24
- mm: represents the minutes
- ss: represents the seconds

This file has a maximum length of 512MB. When the file reaches the maximum size, the PCardio device closes the file and creates a new file where it continues logging data to. Additionally, a new file will be created every time the recorder is reset. This way, when the device is charged in the middle of a measurement, the recording will be split in at least two files: one before charging and one after charging.

The header of the data files contains the copy of the configuration file, and the start time and stop time of the file. The rest of the file corresponds to the data packets. See Section 5 for explanation on how to parse the data packets. When doing so, you need to use the sampling configuration in header of the file.

When downloading the data, make sure to remove the data files after copy them to your local disk, to avoid the memory of the recorder getting full.

Byte Address	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	Config.txt copy															
…																
128																
144	File Start Time				File Stop Time											
…	Data Packets[0:N]															
152 + Packet size * N																

Table 35. Structure of the data files.

8.3 Log file

The log file is automatically generated by the PCardio device. In this file, the recorder logs information that can be of help when the recorder had an unexpected behavior. It is a good practice to copy this file when you download the data and remove it from the recorder, so that it does not grow very large.

9 Input error handling

Accessing device memory for reading or writing can result in invalid operations. When this occurs, the recorder ignores the read or write instruction and notifies the user of the error by updating the input error code and timestamp.

The PCardio device can report 5 different types of errors:

- Read-only error: it occurs when writing to a read-only register.
- Invalid address: this error is reported when the value of the address field is outside the memory map.
- Invalid length: this error happens when the length is bigger than the memory.
- Invalid data: it occurs when the data contains invalid values for any of the registers the user is writing to.
- Invalid state: this error happens when the user tries to modify the measurement configuration while the recorder is logging data.

Error code id	Meaning
0	No error
1	Read-only error
2	Invalid address
3	Invalid length
4	Invalid data
5	Invalid state

Table 36. Error code options.

10 Good practices

10.1 Reading/writing multiple fields

Reading from- and writing to the device memory can be done in bigger memory blocks that span multiple fields in the memory map. However, it is important to note that several memory locations require the PIN to be written, before reading or writing is allowed.

10.2 Single operations

The device can only handle single operations at a time. It is advised to wait for a data transaction to complete, before starting a new transaction. Every packet send to the device should result in one or more packets send from the device. These returned packets could be used to trigger successive transactions. Therefore, it is strongly advised to avoid sending read or data packets while the device is streaming. Instead, send an abort packet, perform the read/write operations you need and then restart the streaming.

10.3 Battery and data size monitoring

It is advised to frequently read the battery level and the size of the data file, to avoid getting to a low power or full storage state.

When the recorder gets into low power mode, it is not responsive, and you need to wait until the battery has charged enough before using it again.

On the other hand, when there is no storage left, the recorder will not start recording, or it will stop an ongoing recording.

10.4 Quick Start

The diagram below shows an example of the steps you need to follow to setup and record a measurement.

As it is shown in the diagram, the first step upon connection is to write to the pin register.

Then, it is important that the time in the recorder is accurate. This is especially important when you want to schedule a measurement. It is recommended to update the time register when there is a mismatch of more than one second.

To schedule a measurement, change the measurement configuration either by changing the config.txt file, or by updating the measurement configuration memory registers via Bluetooth. When the measurement and the data is downloaded, it is recommended to leave only the config.txt file in the recorder, to avoid the SD card from getting full.

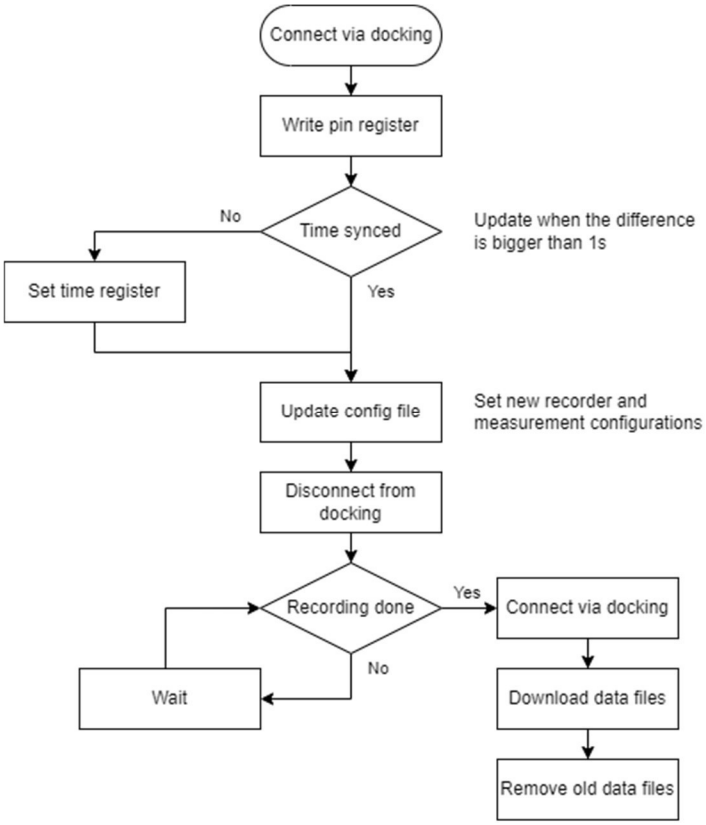


Figure 1. Flow to setup a measurement and record data.

11 Experimental Sensors

The following sensors and sensor data types are experimental and are only available through special firmware and recorder holders.

Sensor	ID	Sensor Gains	Sample Rates	Data Types
PPG	13	0(×1), 1(×2), 2(×3), 4(×6))	3(400Hz), 5(200Hz), 6(100Hz), 7(50Hz)	10
Temperature	14	0(×1)	10(5Hz)	9
ICG	15	0(×1)	0(1000Hz), 1(800Hz), 2(500Hz), 3(400Hz), 4(250Hz), 5(200Hz), 6(100Hz)	11
Heart Rate	16	0(×1)	10(5Hz)	14
Respiration Rate	17	0(×1)	10(5Hz)	15
Heart Rate Variability	18	0(×1)	10(5Hz)	16
Barometer	19	0(×1), 1(×2	10(5Hz)	17
SpO2	20	0(×1)	10(5Hz)	18
Microphone	21	0(x1)	0(1000Hz)	19

Table 37. Experimental sensor Ids and their valid configurations.

Data type ID	Units	Total size (bytes)	Elements	Signed	Analog min	Analog max	Digital min	Digital max
9	Degree Celsius	2	1	Yes	-256	256	-32768	32767
10	Ratio of reflected and transmitted light	4	2 (Red and Infrared channels)	No	0	1365.333	0	65535
11	ADC Counts	3	1	Yes	-524288	524287	-524288	524287
14	Beats per minute	2	1	No	0	300	0	30000
15	Breaths per minute	2	1	No	0	300	0	30000
16	Milliseconds	2	1	No	0	6553.5	0	65535
17	Hectopascal	3	1	Yes	-4096	4096	-8388608	8388607
18	Oxygenation level (%)	2	1	No	0	100	0	10000
19	Sound Envelope (unitless)	2	1	No	0	1	0	65535

Table 38. Experimental data type Ids and their definitions.